

Vorlesung Theoretische Informatik

(05 Reguläre Sprachen)

Alois Heinz
Hochschule Heilbronn,
Max-Planck-Str. 39, 74081 Heilbronn
heinz@hs-heilbronn.de

07/17. April 2026

Motivation für Untersuchungen an Regulären Sprachen

- Die regulären Sprachen über Σ , REG_{Σ} sind zunächst über DFA (NFA & ϵ FA) (akzeptierende Konzepte) definiert worden.
- Ein regulärer Ausdruck beschreibt ein *Muster* für eine Sprache $L \in REG_{\Sigma}$ (beschreibendes Konzept)
- Typ-3-Grammatiken liefern *Regeln* zum Erzeugen der Worte aus $L \in REG_{\Sigma}$ (erzeugendes Konzept)
- Regulären Sprachen besitzen *Abgeschlossenheitseigenschaften* gegenüber gewissen Mengen- und anderen Operationen
- Viele Entscheidungsprobleme für Regulären Sprachen sind effizient entscheidbar
- Regulären Sprachen haben viele Anwendungen in der praktischen Informatik
- Leider sind die Regulären Sprachen aber auch *beschränkt*, bereits sehr einfache Sprachen sind nicht mehr regulär

Reguläre Ausdrücke

Beim Lesen von Worten durch einen NFA werden Operationen durchgeführt, denen hier jetzt Operatoren zugeordnet werden sollen:

- Konkatination von Zeichen, Wörtern: $\bullet$
- Selektion von Wegen: $|$
- Wiederholungen von Zuständen: $\otimes$

Zwei weitere Operatoren sowie Klammern $[,] \notin \Sigma$ werden benötigt:

Der Nulloperator Λ , der Einsoperator: E

Für ein Alphabet Σ ist die Menge der regulären Ausdrücke über Σ $REXP_{\Sigma}$ die Menge der Worte über $\Sigma \cup \{\Lambda, E, \bullet, |, \otimes, [,]\}$, die nach folgenden Regeln gebildet werden:

1. $\Lambda \in REXP_{\Sigma}$
2. $E \in REXP_{\Sigma}$
3. $a \in REXP_{\Sigma}$ für alle $a \in \Sigma$
4. Aus $\alpha, \beta \in REXP_{\Sigma}$ folgt:

$$[\alpha \bullet \beta] \in REXP_{\Sigma}, \quad [\alpha | \beta] \in REXP_{\Sigma}, \quad \alpha^{\otimes} \in REXP_{\Sigma}$$

Semantik Regulärer Ausdrücke

Bsp.: $[0 \bullet [0|1]^\otimes] \in REXP_{\{0,1\}}$, beschrieben werden alle Bitfolgen, die mit 0 beginnen.

Die **Semantik (Bedeutung)** der regulären Ausdrücke ist eine Funktion

$$L : REXP_\Sigma \rightarrow 2^{\Sigma^*}$$

und ist so definiert:

1. $L(\Lambda) = \{ \} = \emptyset$
2. $L(E) = \{ \epsilon \}$
3. $L(a) = \{ a \}$ für alle $a \in \Sigma$
4. Für $\alpha, \beta \in REXP_\Sigma$ gilt:
 - a) $L([\alpha \bullet \beta]) = L(\alpha) \circ L(\beta)$,
 - b) $L([\alpha|\beta]) = L(\alpha) \cup L(\beta)$,
 - c) $L(\alpha^\otimes) = (L(\alpha))^*$

Jetzt: **Vereinfachte Darstellung** von regulären Ausdrücken:

Verwende $\circ$ statt $\bullet$, $*$ statt $\otimes$, ϵ statt E , $\emptyset$ statt Λ , runde statt eckiger Klammern;
 $\circ$ kann auch weggelassen werden. $[0 \bullet [0|1]^\otimes] \in REXP_{\{0,1\}}$ wird zu $(0(0|1)^*)$

Rechenregeln für Reguläre Ausdrücke

- | | | |
|-----|---|---|
| 1) | $(\emptyset \alpha) \equiv (\alpha \emptyset) \equiv \alpha$ | $\emptyset$: Identität für $ $ |
| 2) | $(\epsilon \circ \alpha) \equiv (\alpha \circ \epsilon) \equiv \alpha$ | ϵ : Identität für $\circ$ |
| 3) | $(\emptyset \circ \alpha) \equiv (\alpha \circ \emptyset) \equiv \emptyset$ | $\emptyset$: Nullelement für $\circ$ |
| 4) | $(\alpha \beta) \equiv (\beta \alpha)$ | $ $ ist kommutativ |
| 5) | $((\alpha \beta) \gamma) \equiv (\alpha (\beta \gamma))$ | $ $ ist assoziativ |
| 6) | $((\alpha \circ \beta) \circ \gamma) \equiv (\alpha \circ (\beta \circ \gamma))$ | $\circ$ ist assoziativ — nicht kommutativ! |
| 7) | $((\alpha \beta) \circ \gamma) \equiv ((\alpha \circ \gamma) (\beta \circ \gamma))$ | |
| | $(\alpha \circ (\beta \gamma)) \equiv ((\alpha \circ \beta) (\alpha \circ \gamma))$ | Distributivgesetze |
| 8) | $(\alpha \alpha) \equiv \alpha$ | Idempotenzgesetz für $ $ |
| 9) | $\emptyset^* \equiv \epsilon$ | |
| 10) | $(\alpha \circ (\alpha)^*) \equiv ((\alpha)^* \circ \alpha)$ | $(\alpha \circ (\alpha)^*) \equiv \alpha^+$ |
| 11) | $((\alpha)^+ \epsilon) \equiv (\alpha)^*$ | |

Die Rechenregeln helfen beim Vereinfachen (äquivalenten Umformen) von Regulären Ausdrücken.

Beispiel für Anwendung Regulärer Ausdrücke

Anwendung:

Erkenne korrekte Angaben zu Geldtransfers zwischen Konten. Mögliche Währungsangaben: USD, EUR, ATS. Danach Betrag: +, - optional, dann ganzzahliger Teil, Punkt, Dezimalteil. Ganzzahliger Teil: beliebig lang, ohne führende Null. Dezimalteil: genau 2 Stellen. Punkt und Dezimalteil zusammen optional.

Daraus ergibt sich:

transfer = währung ◦ vorzeichen ◦ betrag
währung = (USD|EUR|ATS)
vorzeichen = (+| - |€)
betrag = ganz ◦ (€|. ◦ dez)
ganz = (0|(1|2|3|4|5|6|7|8|9) ◦ (0|1|2|3|4|5|6|7|8|9)*)
dez = (0|1|2|3|4|5|6|7|8|9) ◦ (0|1|2|3|4|5|6|7|8|9)

Somit: transfer = (USD|EUR|ATS) ◦ (+| - |€) ◦ (0|(1|2|3|4|5|6|7|8|9) ◦ (0|1|2|3|4|5|6|7|8|9)*) ◦ (€|. ◦ (0|1|2|3|4|5|6|7|8|9) ◦ (0|1|2|3|4|5|6|7|8|9))

Reguläre Ausdrücke für Automaten

Satz. Zu jedem endlichen Automaten A über Σ gibt es einen regulären Ausdruck α über Σ , so dass $L(A) = L(\alpha)$ gilt. ($DFA_{\Sigma} \subseteq LREXP_{\Sigma}$)

Beweis. Sei $A = (\Sigma, \{s_1, \dots, s_n\}, \delta, s_1, F)$. Es werden rekursiv (Beschreibungen für) $R(i, j, k) \subseteq \Sigma^*$ konstruiert mit Wörtern, die in A von s_i nach s_j führen und nur Zwischen-Zustände s_q mit $q \leq k$ berühren:

1. $R(i, j, 0) = \{a \mid \delta(s_i, a) = s_j\}$, falls $i \neq j$, und $R(i, j, 0) = \{a \mid \delta(s_i, a) = s_j\} \cup \{\epsilon\}$ sonst
2. Für $k > 0$ gilt:
$$R(i, j, k) = R(i, j, k-1) \cup R(i, k, k-1) \circ R(k, k, k-1)^* \circ R(k, j, k-1)$$

Schließlich gilt:

$$L(A) = \bigcup_{s_j \in F} R(1, j, n)$$

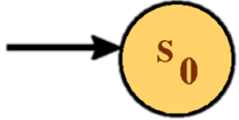
□

Automaten für Reguläre Ausdrücke

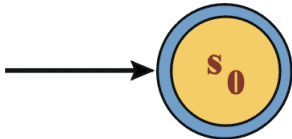
Satz. Zu jedem regulären Ausdruck α über Σ , gibt es einen endlichen Automaten A über Σ so dass $L(A) = L(\alpha)$ gilt. ($DFA_{\Sigma} \supseteq LREXP_{\Sigma}$)

Der Beweis erfolgt per Induktion über den Aufbau regulärer Ausdrücke:

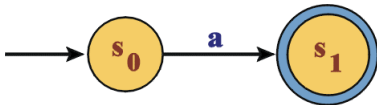
- $\emptyset$ wird erkannt:



- $\{\epsilon\}$ wird erkannt:

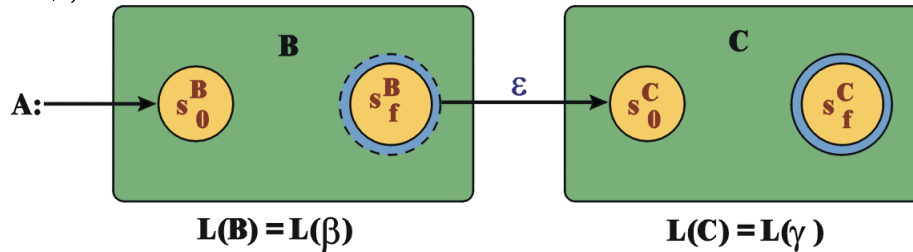


- $\{a\}$ wird erkannt:

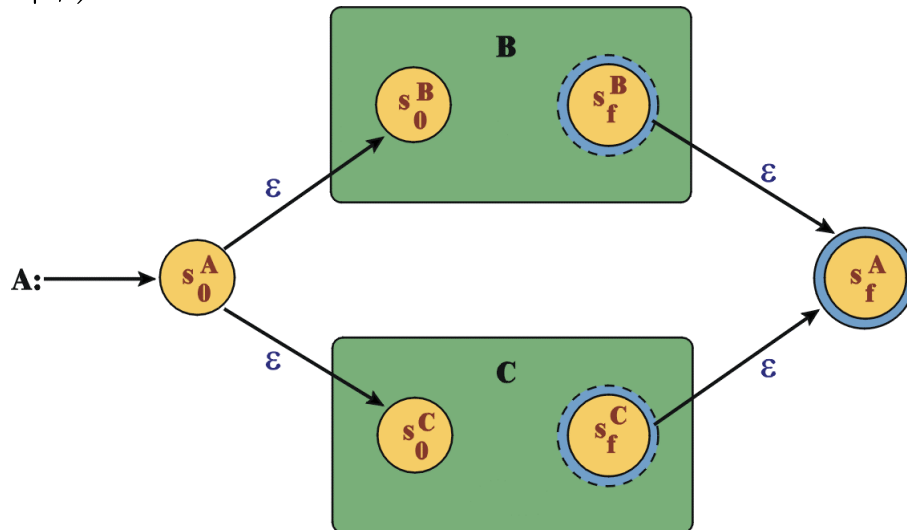


Automaten für Reguläre Ausdrücke (2)

- $L(\beta \gamma)$ wird erkannt:

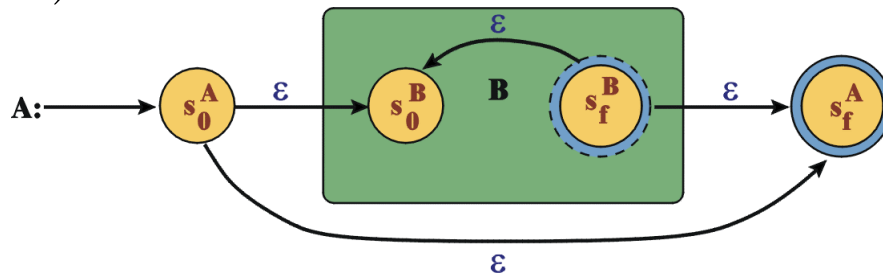


- $L(\beta|\gamma)$ wird erkannt:



Automaten für Reguläre Ausdrücke (3)

- $L(\beta^*)$ wird erkannt:

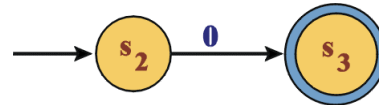
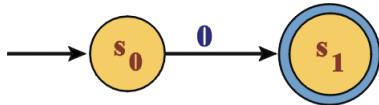


Satz. Die durch reguläre Ausdrücke über Σ beschreibbaren Sprachen sind gleich den durch endliche Automaten erkennbaren Sprachen über Σ .
($DFA_\Sigma = LREXP_\Sigma = REG_\Sigma$)

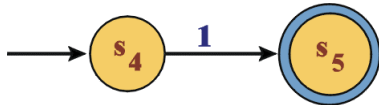
Beispiel: Automaten für einen Regulären Ausdruck (1)

Aufgabe: Baue nach Konstruktionsanleitung Automaten für $(0(0|1)^*)$

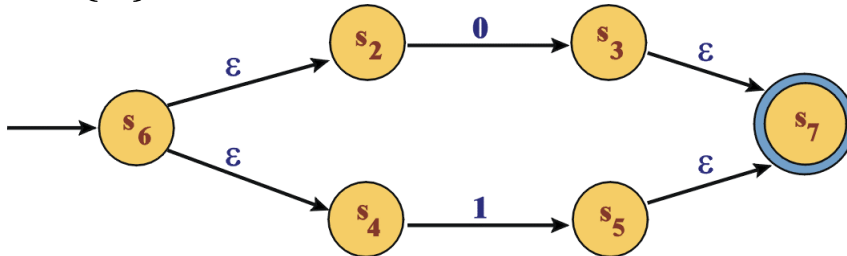
- $\{0\}$ wird erkannt:



- $\{1\}$ wird erkannt:

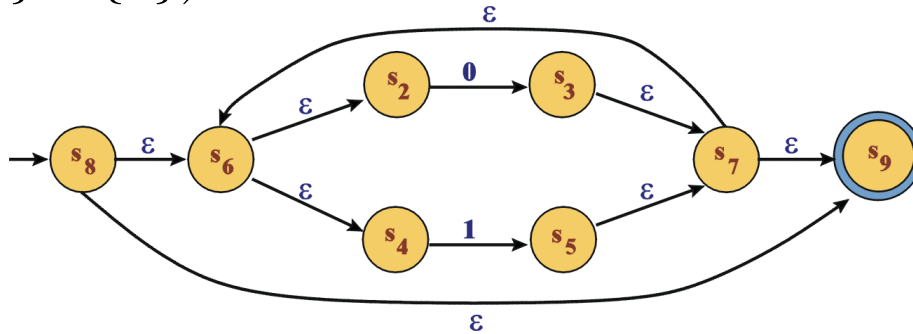


- $\{0\} \cup \{1\}$ wird erkannt:

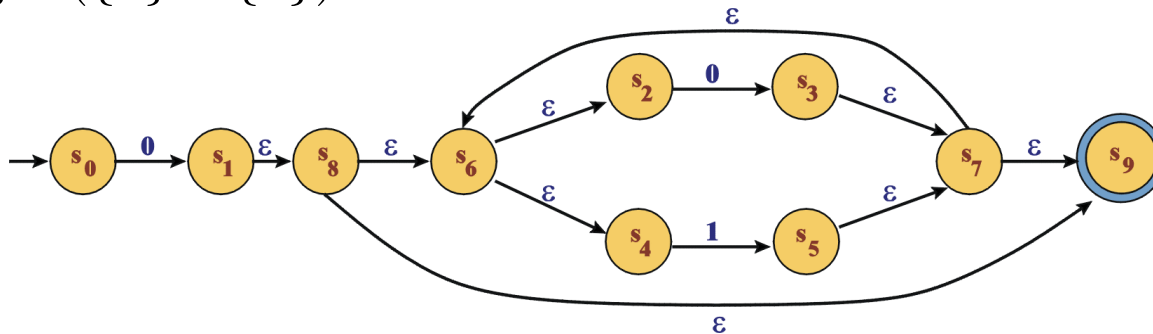


Beispiel: Automaten für einen Regulären Ausdruck (2)

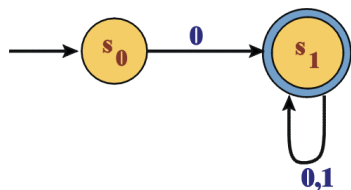
- $(\{0\} \cup \{1\})^*$ wird erkannt:



- $\{0\} \circ (\{0\} \cup \{1\})^*$ wird erkannt:



- Derselbe Automat nach ϵ -Elimination:



Typ-3-Grammatiken

Sei Σ ein Alphabet. Eine rechtslineare Grammatik

$$G = (\Sigma, N, P, S)$$

über Σ besteht aus dem Terminalalphabet Σ , dem davon disjunkten Nonterminalalphabet N , einer endlichen Relation $P \subseteq N \times (\Sigma N \cup \Sigma \cup \{\epsilon\})$ sowie dem Startsymbol $S \in N$

Dabei sind die Elemente $p = (\ell, r) \in P$ Regeln oder Produktionen, sie werden oft so notiert: $\ell \rightarrow r$ (ℓ geht über in r).

Bsp: $G = (\{0, 1\}, \{S, A, B\}, P, S)$ mit

$$P = \{S \rightarrow 0S, S \rightarrow 1S, S \rightarrow 0A, A \rightarrow 0B, A \rightarrow 1B, B \rightarrow 0, B \rightarrow 1\}$$

ist eine rechtslineare Grammatik

Ableitung, Sprache einer Grammatik

Ein **Ableitungsschritt** einer rechtslinearen Grammatik $G = (\Sigma, N, P, S)$ ist eine Relation

$$\Rightarrow \subseteq \Sigma^* N \times \Sigma^*(N \cup \{\epsilon\})$$

definiert durch $uA \Rightarrow uw$ genau dann, wenn $A \rightarrow w \in P$, mit $u \in \Sigma^*$, $A \in N$,
 $w \in \Sigma N \cup \Sigma \cup \{\epsilon\}$

Die **Sprache** $L(G)$ einer Grammatik G kann nun mit Hilfe der transitiv-reflexiven Hülle $\Rightarrow^*$ von $\Rightarrow$ definiert werden:

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$$

Ein Wort in $L(G)$ besteht nur aus Terminalzeichen und kann in endlich vielen Schritten aus S abgeleitet werden.

Bsp: $S \Rightarrow 0S \Rightarrow 01S \Rightarrow 011S \Rightarrow 0110A \Rightarrow 01101B \Rightarrow 011010$ oder kurz $S \Rightarrow^* 011010$

Rechtslineare Sprachen, Grammatik-Vereinfachung

$L \subseteq \Sigma^*$ ist rechtslinear, wenn sie von einer rechtslinearen Grammatik erzeugt werden kann.
 ReL_Σ sei die Klasse der rechtslinearen Sprachen.

Die drei Arten der zugelassenen Regeln für rechtslineare Grammatiken:

1. $A \rightarrow aB$
2. $A \rightarrow a$
3. $A \rightarrow \epsilon$

Grammatik-Vereinfachung: Es reichen Regeln der Art 1. und 3.: Jede Regel $A \rightarrow a$ kann durch Regeln $A \rightarrow aC$ und $C \rightarrow \epsilon$ für ein neues $C \in N$ ersetzt werden.

Bsp: $P = \{S \rightarrow 0S, S \rightarrow 1S, S \rightarrow 0A, A \rightarrow 0B, A \rightarrow 1B, B \rightarrow 0, B \rightarrow 1\}$
kann ersetzt werden durch

$$P = \{S \rightarrow 0S, S \rightarrow 1S, S \rightarrow 0A, A \rightarrow 0B, A \rightarrow 1B, \\ B \rightarrow 0C_0, B \rightarrow 1C_1, C_0 \rightarrow \epsilon, C_1 \rightarrow \epsilon\}$$

Bem.: Bis auf $S \rightarrow \epsilon$ kommt man mit einer einzigen Regel der Art 3. aus: $T \rightarrow \epsilon$

Linkslinere Grammatiken und Sprachen

Sei Σ ein Alphabet. Eine linksslinere Grammatik

$$G = (\Sigma, N, P, S)$$

über Σ besteht aus dem Terminalalphabet Σ , dem davon disjunkten Nonterminalalphabet N , einer endlichen Relation $P \subseteq N \times (N\Sigma \cup \Sigma \cup \{\epsilon\})$ sowie dem Startsymbol $S \in N$.

Ableitungsschritte und die Klasse der linksslineren Sprachen LiL_Σ sind analog zu den rechtslineren Formen definiert.

Satz. Für beliebiges Alphabet Σ gilt: $ReL_\Sigma = LiL_\Sigma$. Zu jeder rechtslineren Grammatik G existiert eine äquivalente linkslinere Grammatik G' und umgekehrt.

Beweis.[Skizze]: $\subseteq$: Gegeben sei eine rechtslinere Grammatik G mit einer ϵ -Regel $T \rightarrow \epsilon$. T wird Startsymbol in Grammatik G' , alle Regeln $A \rightarrow aB$ aus G werden transformiert zu $B \rightarrow Aa$ in G' , weiter kommt $S \rightarrow \epsilon$ zu G' und außerdem noch $T \rightarrow \epsilon$, falls $S \rightarrow \epsilon$ in G ist. $\supseteq$: analog $\square$

$TYP3_\Sigma := ReL_\Sigma = LiL_\Sigma$ Man spricht von Typ-3-Sprachen und Typ-3-Grammatiken.

Äquivalenz von Typ-3-Grammatiken und DFA (1)

Satz. Zu jeder Typ-3-Grammatik G über Σ gibt es einen DFA A über Σ mit $L(G) = L(A)$

Beweis. Gegeben sei eine rechtslineare Grammatik $G = (\Sigma, N, P, S)$.
Daraus wird zunächst ein NFA A konstruiert mit

$$A = (\Sigma, N \cup \{f\}, \delta, S, F) \text{ und}$$

$$F = \{B \in N \mid B \rightarrow \epsilon \in P\} \cup \{f\}$$

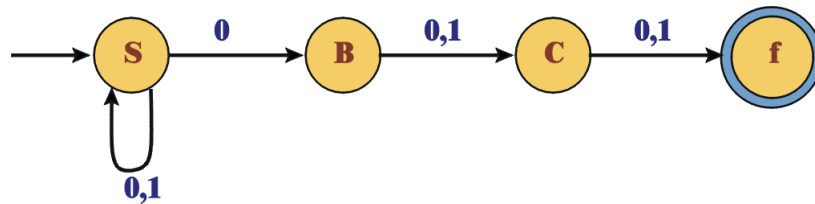
$$\delta = \{(B, a, C) \mid B \rightarrow aC \in P\} \cup \{(B, a, f) \mid B \rightarrow a \in P\}$$

Zu diesem NFA kann nun noch ein äquivalenter DFA konstruiert werden. $\square$

Bsp: $G = (\{0, 1\}, \{S, B, C\}, P, S)$ mit

$$P = \{S \rightarrow 0S, S \rightarrow 1S, S \rightarrow 0B, B \rightarrow 0C, B \rightarrow 1C, C \rightarrow 0, C \rightarrow 1\}$$

liefert NFA:



Äquivalenz von Typ-3-Grammatiken und DFA (2)

Satz. Zu jedem endlichen Automaten A über Σ gibt es eine rechtslineare Grammatik G über Σ mit $L(G) = L(A)$

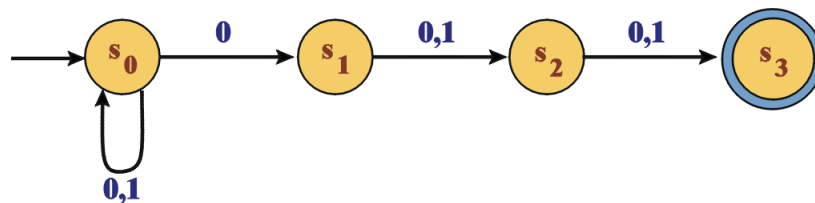
Beweis. Gegeben sei ein NFA $A = (\Sigma, S, \delta, s_0, F)$. Daraus wird eine rechtslineare Grammatik G konstruiert mit

$$G = (\Sigma, S, P, s_0) \text{ und}$$

$$P = \{s \rightarrow at \mid \delta(s, a) = t\} \cup \{s \rightarrow \epsilon \mid s \in F\}$$

□

Bsp: Für den NFA



erhält man die Grammatik $G = (\{0, 1\}, \{s_0, s_1, s_2, s_3\}, P, s_0)$ mit

$$P =$$

$$\{s_0 \rightarrow 0s_0, s_0 \rightarrow 1s_0, s_0 \rightarrow 0s_1, s_1 \rightarrow 0s_2, s_1 \rightarrow 1s_2, s_2 \rightarrow 0s_3, s_2 \rightarrow 1s_3, s_3 \rightarrow \epsilon\}$$

Abschlusseigenschaften regulärer Sprachen (1)

Satz. Für jedes Alphabet Σ gilt:

1. $\emptyset \in REG_{\Sigma}$
2. $\Sigma \in REG_{\Sigma}$
3. $\Sigma^* \in REG_{\Sigma}$
4. $\{w\} \in REG_{\Sigma}$ für jedes $w \in \Sigma^*$
5. $L \in REG_{\Sigma}$ für jedes $L \subseteq \Sigma^*$ mit $|L| < \infty$

Beweis. Übung. $\square$

Abschlusseigenschaften regulärer Sprachen (2)

Satz. Für ein Alphabet Σ und $L, L_1, L_2 \in REG_\Sigma$ gilt:

1. $L_1 \cup L_2 \in REG_\Sigma$, Vereinigung
2. $L_1 \setminus L_2 \in REG_\Sigma$, Differenz
3. $\Sigma^* \setminus L \in REG_\Sigma$, Komplement
4. $L_1 \cap L_2 \in REG_\Sigma$, Durchschnitt
5. $L_1 \circ L_2 \in REG_\Sigma$, Konkatenation
6. $L^* \in REG_\Sigma$, Kleene-Stern-Produkt
7. $SP(L) \in REG_\Sigma$, Spiegelung

Beweis. Übung. $\square$